SkillNavora

MODULE 3 HANDBOOK

Module 3: Artificial Intelligence and Machine Learning

4 sections, every topic explained with how it works, examples and resources.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.

Financial figures are hypothetical examples, not advice.

Module 3: Artificial Intelligence and Machine Learning

Every section below explains the concept, how it works, each listed topic in detail, a worked example and where to learn more.

Sections

1. 3.1 AI Fundamentals
2. 3.2 Machine Learning
3. 3.3 Model Evaluation
4. 3.4 AI Model Deployment

MODULE 3 · SECTION 3.1

3.1 AI Fundamentals

DEFINITION

> Artificial intelligence is software that performs tasks that seem to need human judgement. Machine learning learns patterns from data instead of following hand-written rules. Deep learning uses many-layered neural networks. Generative AI produces new content.

HOW IT WORKS

- Training adjusts a model's parameters (weights) to reduce error on example data. Inference uses the trained model on new input.
- Features are input measurements; labels are the correct answers in supervised data.
- Models have limits: they reflect their training data, can be wrong with confidence, and do not know events after training unless given that information.

EVERY TOPIC EXPLAINED

AI vs ML vs deep learning; generative vs traditional AI. AI is the broad goal of machines doing tasks that need intelligence. Machine learning is the main method: learning patterns from data. Deep learning uses multi-layer neural networks and powers vision, speech and LLMs. Traditional AI predicts or classifies (fraud yes/no); generative AI creates new content (text, images, code).

Neural networks; training vs inference; parameters. A neural network has layers of connected units; each connection has a weight. Training shows the network examples, measures the error with a loss function, and adjusts weights using gradient descent. Inference runs the trained network on new input; it is what happens each time your app calls a model. Parameters are the learned weights, often billions in LLMs.

Features, labels, datasets, model limitations. Features are the input variables (age, income, words); labels are the answers in training data. A dataset is split into training, validation and test sets. Limitations: models reflect bias in data, fail on data unlike their training, cannot explain themselves fully, and may be confidently wrong.

WORKED EXAMPLE

> A spam filter learns from labelled emails (features: words, sender; label: spam or not) and is then used on new mail (inference).

RESOURCES

- Google Machine Learning Crash Course https://developers.google.com/machine-learning/crash-course
- Stanford CS229 notes https://cs229.stanford.edu/

3.2 Machine Learning

DEFINITION

> Supervised learning learns from labelled examples. Unsupervised learning finds structure without labels. Reinforcement learning learns from rewards.

HOW IT WORKS

- Classification predicts a category; regression predicts a number; clustering groups similar items.
- Decision trees split data by rules; random forests average many trees; gradient boosting builds trees that correct earlier ones; neural networks learn layered transformations.
- Feature engineering turns raw data into useful inputs. The data is split into training, validation and test sets so the model is judged on data it has not seen.

EVERY TOPIC EXPLAINED

Supervised, unsupervised and reinforcement learning. Supervised learning uses labelled examples (emails marked spam). Unsupervised learning finds structure in unlabelled data (customer segments). Reinforcement learning learns by trial and reward (game playing, and fine-tuning LLMs with human feedback).

Classification, regression, clustering. Classification predicts a category (churn: yes/no). Regression predicts a number (house price). Clustering groups similar items without labels (k-means).

Decision trees, random forests, gradient boosting, neural networks. Decision tree: a flowchart of if-then splits; easy to explain. Random forest: many trees vote; more accurate and stable. Gradient boosting (XGBoost, LightGBM): trees added one after another to fix previous errors; often the best for tabular data. Neural networks: layered models for images, text and audio.

Feature engineering, preprocessing, training, validation, testing. Feature engineering creates useful inputs (days since last purchase). Preprocessing scales numbers, encodes categories and fills missing values. Training fits the model; validation tunes settings; testing gives the final unbiased score on data never used before.

WORKED EXAMPLE

> A bank predicts loan default (classification) from income and history features, trained on past loans and tested on held-out loans.

RESOURCES

- scikit-learn user guide (official) https://scikit-learn.org/stable/user_guide.html
- Google ML Crash Course https://developers.google.com/machine-learning/crash-course

3.3 Model Evaluation

DEFINITION

> Evaluation measures how well a model performs on unseen data, using metrics that match the business cost of errors.

HOW IT WORKS

- Accuracy is correct predictions divided by all. Precision is how many flagged items were right; recall is how many real items were found; F1 balances them.
- The confusion matrix counts true and false positives and negatives. ROC-AUC summarises performance across thresholds.
- Regression uses MAE (average absolute error) and MSE (squared error).
- Overfitting = memorising training data; underfitting = too simple. Cross-validation tests on several splits. Model drift = performance falls as the world changes.

EVERY TOPIC EXPLAINED

Accuracy, precision, recall, F1, confusion matrix, ROC-AUC. Accuracy = correct ÷ total. Precision = true positives ÷ predicted positives (when it says yes, how often is it right). Recall = true positives ÷ actual positives (how many real cases it finds). F1 = harmonic mean of precision and recall. Confusion matrix: TP, FP, TN, FN counts. ROC-AUC: how well the model ranks positives above negatives across all thresholds (0.5 random, 1.0 perfect).

MAE, MSE; overfitting, underfitting, cross-validation, bias and variance, model drift. MAE = average absolute error; MSE = average squared error (punishes large errors). Overfitting: great on training, poor on new data. Underfitting: poor everywhere. Cross-validation trains and tests on several splits for a reliable estimate. Bias = systematic error from a too-simple model; variance = sensitivity to the training sample. Model drift: data or behaviour changes over time and accuracy falls; detect it with monitoring.

WORKED EXAMPLE

> A fraud model with 99% accuracy can still miss every fraud case if fraud is 1% of data; recall on fraud is the metric that matters.

RESOURCES

- scikit-learn: model evaluation https://scikit-learn.org/stable/modules/model_evaluation.html
- Google ML Crash Course: classification metrics https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall

3.4 AI Model Deployment

DEFINITION

> Deployment makes a trained model available to applications, usually as an inference API, with monitoring and version control.

HOW IT WORKS

- Serve the model behind an HTTP endpoint (for example FastAPI). Batch inference scores many rows on a schedule; real-time inference answers each request.
- Version each model and keep the old one until the new one proves itself. Monitor latency, errors and output drift.
- Retrain when monitored quality drops below an agreed threshold.

EVERY TOPIC EXPLAINED

Model serving; inference APIs; batch and real-time inference. Model serving makes a model available to other software, usually as a REST endpoint returning predictions. Inference APIs (OpenAI, Claude, Vertex AI, SageMaker) host the model for you. Batch inference scores large sets on a schedule; real-time inference answers each request in milliseconds to seconds.

Model monitoring, versioning and retraining; production evaluation. Monitoring tracks prediction volume, latency, errors, input data distribution and accuracy when labels arrive. Versioning gives each model and its training data an ID so you can roll back. Retraining refreshes the model on new data when drift is detected. Production evaluation compares live results with the offline test score.

WORKED EXAMPLE

> A scoring model is wrapped in a FastAPI route, containerised with Docker, and deployed behind a load balancer; a dashboard tracks prediction drift weekly.

RESOURCES

- FastAPI documentation (official) https://fastapi.tiangolo.com/
- Docker: get started (official) https://docs.docker.com/get-started/